PIMS Technical Specification

Product Information Management System, final MVP spec

1. Purpose and scope

PIMS is the system of record for product data. WooCommerce is the storefront. PIMS creates and updates WooCommerce products through the WooCommerce REST API and can import existing WooCommerce products.

In scope (MVP): product catalog CRUD, categories, dynamic attributes, product-level sync with field-level conflict handling, sync status and logging, authentication.

Out of scope (MVP): product variations, images and media, orders and customers, scheduled or bulk sync (other than import), webhooks, category hierarchy, multi-store, multi-language, user roles.

2. Decisions

Topic	Decision
Source of truth	PIMS, except stock, which WooCommerce owns after the first push.
Stack	PHP 8, MySQL 8.0.16+, HTML, CSS and vanilla JavaScript.
Architecture	API-first: JSON API with thin controllers, a service layer for business logic, and a plain JS frontend calling the API.
Auth	PHP sessions, one user role, CSRF protection.
Sync trigger	Manual, per product, plus one bulk import action. No background jobs.
Conflict handling	Field-level three-way compare against a stored snapshot; resolved per field, with keep-all shortcuts.

Topic	Decision
Attributes	`select` attributes map to WooCommerce global attributes; all other types are sent as custom per-product attributes.
Delete	Never-synced products are hard deleted; synced products are archived.
Categories	Flat (no parent/child).
Brand	Free-text field (WooCommerce has no native brand).
Concurrent edits	Optimistic locking via a `version` column.

3. Architecture

```
Browser (HTML / CSS / JS)
        | fetch + session cookie + CSRF token
        v
JSON API (thin PHP controllers)
        |
        v
Services: CatalogService, AttributeService, SyncService
        |                       |
        v                       v
Repositories (PDO)       WooCommerceClient (cURL)
        |                       |
        v                       v
     MySQL            WooCommerce REST API v3
```

Controllers validate input and call services. They contain no business logic. Services own all rules (validation, merge, status transitions) so the UI and API cannot disagree. If time runs short, server-rendered pages can call the same services without changing them.

4. Functional requirements

4.1 Product catalog

Field	Type	Rules
id	integer, PK	Generated by PIMS
sku	string	Required, unique, immutable after first sync
name	string	Required, max 255
description	text	Optional
category_id	FK	Required
brand	string	Optional
price	decimal(10,2)	Required, >= 0
stock	integer	Required, >= 0. Editable until the first push; read-only afterward (see 4.3).
status	enum	`draft`, `active`, `inactive`, `archived`
version	integer	Incremented on every update; used for optimistic locking
created_at, updated_at	timestamp	Maintained by PIMS

Operations: create, view, edit, archive or delete, restore, search (name or SKU), filter (category, brand, status), paginate. Archived products are hidden by default.

Optimistic locking: an update request carries the `version` it was based on. If the stored version differs, the update is rejected with a 409 and the user reloads.

Delete rules:

- A product with no `woocommerce_id` is hard deleted.
- A product with a `woocommerce_id` is archived. Archiving marks it `NOT_SYNCED`, and the next sync sets it to trash in WooCommerce.
- An archived product can be permanently deleted only after a successful sync confirms it is trashed in WooCommerce.

- Import requests exclude trashed WooCommerce products, so deleted products do not reappear.

4.2 Attributes

Attributes are defined once, assigned to categories, and given values per product.

- **Data types:** `text`, `number`, `boolean`, `select`, `unit` (a number plus a fixed unit defined on the attribute, e.g. 16 GB).
- `select` attributes own a list of allowed options.
- A category can mark an attribute as required. A product in that category cannot become `active` while a required attribute has no value.
- An attribute's data type cannot change once any product has a value for it. An attribute with values cannot be deleted until those values are removed.
- Changing a product's category keeps values for attributes shared with the new category and asks before discarding the rest.

4.3 Stock ownership

WooCommerce owns stock because it changes with every order, which PIMS cannot see.

- Stock is editable in PIMS only until the product's first push. That value becomes the initial WooCommerce inventory, with stock management enabled.
- After the first push, stock is read-only in PIMS.
- Sync never pushes stock after the first push. Every sync overwrites the PIMS value from WooCommerce, with no conflict check.
- Stock is excluded from conflict detection and never moves a product to `NOT_SYNCED`.
- The product page has a refresh action so users can pull current stock on demand.

4.4 Synchronization

Synced fields: name, description, price, category, status, attributes (plus stock under the rules in 4.3).

PIMS	WooCommerce
sku	sku
name	name

PIMS	WooCommerce
description	description
price	regular_price
stock	stock_quantity (first push only; manage_stock = true)
category	categories, via `categories.woocommerce_id` (created in WooCommerce on first use)
attributes	`select`: global attribute and term IDs. Other types: custom attributes.
status: active / draft / inactive / archived	publish / draft / private / trash

Product sync is a three-way merge per field, comparing the PIMS value, the WooCommerce value, and the last-synced snapshot (`wc_snapshot`):

Snapshot	PIMS	WooCommerce	Result
same	same	same	No change
same	changed	same	Push the PIMS value
same	same	changed	Pull the WooCommerce value
same	changed	changed, different	**Conflict** on this field
same	changed	changed, identical	No change; snapshot updated

If the product has no `woocommerce_id`, sync first looks up WooCommerce by SKU. If found, it links the product and merges. If not found, it creates the product. A product with any conflicting field becomes `CONFLICT`, and no fields are written for that product until the conflict is resolved.

Select attributes: on first use, PIMS creates the attribute in WooCommerce (`products/attributes`) and each option as a term, storing the returned IDs on `attributes.woocommerce_id` and `attribute_options.woocommerce_term_id`. A name or slug collision links to the existing WooCommerce attribute instead of creating a duplicate.

Import fetches WooCommerce products page by page, excluding trashed ones. Each is matched by `woocommerce_id`, then SKU. Unmatched products are created in PIMS as `draft`. Matched products go through the same three-way merge as a product sync. WooCommerce products without a SKU are skipped and logged.

Conflict resolution. The conflict view lists each conflicting field with the PIMS and WooCommerce values. The user chooses a value per field, or uses a keep-all-PIMS or keep-all-WooCommerce shortcut. After resolution the sync is re-run with the chosen values, and on success the status becomes `SYNCED`.

Sync status:

Status	Meaning
NOT_SYNCED	Never synced, or PIMS edited since the last sync
SYNCING	In progress; blocks a second sync of the same product
SYNCED	PIMS and WooCommerce match as of `last_synced_at`
FAILED	Last attempt errored; see log
CONFLICT	Fields changed on both sides; awaiting resolution

A `SYNCING` state older than 5 minutes is treated as `FAILED`. Every attempt writes a `sync_logs` row (product, direction, outcome, error, WooCommerce HTTP status, timestamp). Conflicting fields and both values are stored in `products.conflict_fields` while the product is in `CONFLICT`.

5. Data model

```
products            id, sku (unique), name, description, category_id, brand,
                    price, stock, status, version, woocommerce_id (unique,
nullable),
                    sync_status, last_synced_at, last_sync_direction,
                    last_sync_error, wc_snapshot (JSON), conflict_fields
(JSON),
                    created_at, updated_at
categories          id, name (unique), description, woocommerce_id
(nullable),
                    created_at, updated_at
attributes          id, name (unique), data_type, unit (nullable),
                    woocommerce_id (nullable), created_at, updated_at
attribute_options   id, attribute_id, value, woocommerce_term_id (nullable)
category_attributes category_id, attribute_id, required   -- PK (category_id,
attribute_id)
product_attributes  product_id, attribute_id, value_text, value_number,
                    value_boolean, option_id              -- PK (product_id,
attribute_id)
sync_logs           id, product_id (nullable), direction, outcome, error,
                    http_status, created_at
users               id, email (unique), password_hash, created_at
```

- `product_attributes` uses typed columns so type validation and numeric filtering work. Exactly one value column is populated, determined by the attribute's `data_type`. Enforce this in the service layer; a CHECK constraint also works on MySQL 8.0.16 and later, but older versions silently ignore it.
- Use InnoDB and utf8mb4 throughout.
- Required indexes: `products(sku)`, `products(category_id, status)`, `products(woocommerce_id)`.
- `sync_logs.product_id` is nullable so logs survive a hard delete.

6. API

All endpoints require an authenticated session except login. State-changing requests must include a CSRF token header. List endpoints accept `page` and `limit`. Errors return `{ error: { code, message, fields } }`.

Area	Endpoints
Auth	`POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`
Products	`GET /api/products` (search, filter), `GET /api/products/{id}`, `POST /api/products`, `PUT /api/products/{id}`, `DELETE /api/products/{id}` (hard delete or archive per 4.1), `POST /api/products/{id}/restore`
Categories	`GET`, `POST`, `PUT /{id}`, `DELETE /{id}` (blocked while products exist), `PUT /api/categories/{id}/attributes` (replaces the assigned set with required flags)
Attributes	`GET`, `POST`, `PUT /{id}`, `DELETE /{id}`, plus option management for `select` types
Sync	`POST /api/products/{id}/sync`, `GET /api/products/{id}/sync-status`, `POST /api/products/{id}/sync/resolve`, `POST /api/products/{id}/refresh-stock`, `POST /api/sync/import`, `GET /api/sync/logs`

`PUT /api/products/{id}` accepts product fields and attribute values together and saves them in one transaction. `sync/resolve` takes either a per-field choice map (each conflicting field set to `pims` or `woocommerce`) or a single keep-all choice.

7. Validation and error handling

- Required fields present; SKU unique (409 on duplicate); price and stock numeric and non-negative.
- Attribute values match the attribute's type; `select` values are allowed options; required attributes enforced before `active`.
- WooCommerce calls use a 10 second timeout. Retry once on a network error or 5xx. Do not retry on 4xx.
- Malformed or non-JSON WooCommerce responses mark the sync `FAILED` and log the status and a truncated body.

- `woocommerce_id` is unique, so one WooCommerce product can never be mapped twice.
- A failed sync never leaves partial writes on a product: PIMS-side changes from a merge are applied in one transaction after the WooCommerce calls succeed.

8. Security

- Passwords hashed with `password_hash`. Session cookie flagged `HttpOnly`, `Secure`, and `SameSite=Lax`; session ID regenerated on login.
- CSRF token required on every state-changing request.
- All SQL through PDO prepared statements; server-side validation on every write; output escaped in the UI.
- WooCommerce consumer key and secret live in a config or environment file outside the web root. They are never returned by any endpoint, never logged, and never reachable from the browser.
- WooCommerce traffic over HTTPS only.

9. Build order

1. Schema, auth, product and category CRUD, optimistic locking.
2. Attribute definitions, category assignment, product values, validation.
3. Frontend: product list, product form with category-driven attribute fields.
4. WooCommerce client, product sync (create and merge), stock rules, status and logging.
5. Global attribute mapping for `select` types.
6. Import, conflict detection and per-field resolution.

Steps 1 to 4 give a demoable build. Steps 5 and 6 are the stretch, with step 6 first if only one fits.

10. Acceptance criteria

1. A user can create a Laptop category with Processor (text), RAM (unit), and Color (select) attributes, then create a laptop with a value for each.
2. A product cannot become `active` while a required attribute is empty.

3. Syncing a new product creates it in WooCommerce and stores its `woocommerce_id`; syncing again updates it and creates no duplicate.
4. A WooCommerce product with a matching SKU is linked, not duplicated.
5. Importing brings in unmapped products as drafts and skips products without SKUs.
6. Changing the price in PIMS and the description in WooCommerce, then syncing, updates both sides with no conflict.
7. Changing the same field to different values on both sides produces `CONFLICT`, writes nothing, and can be resolved per field.
8. After a WooCommerce sale, syncing updates PIMS stock and never overwrites the WooCommerce count.
9. A forced WooCommerce failure produces `FAILED`, a readable error on the product, and a `sync_logs` row.
10. Two users editing the same product: the second save is rejected with a version conflict.
11. No WooCommerce credential appears in any API response, frontend code, or log.

11. Deferred

Webhooks for WooCommerce-to-PIMS changes, variations, images, scheduled sync, category hierarchy, user roles, and global mapping for non-select attribute types.